

Learning Transformation Rules for Semantic Query Optimization: A Data-Driven Approach

Shashi Shekhar
Babak Hamidzadeh
Ashim Kohli
Mark Coyle

Computer Science Dept.
University of Minnesota
Minneapolis, MN 55455

ABSTRACT

Learning query transformation rules is vital for the success of semantic query optimization in domains where the user cannot provide a comprehensive set of integrity constraints. Finding these rules is a discovery task because of the lack of target. Previous approaches to learning query transformation rules have been based on analyzing past queries. We propose a new approach to learning query transformation rules based on analyzing the existing data in the database. This paper describes a framework and a closure algorithm to learning rules from a given data-distribution. We characterize the correctness, completeness and complexity of the proposed algorithm and provide a detailed example to illustrate the framework.

Keywords: Rule discovery, semantic query optimization, discovery in data.

Areas Addressed: Learning and Discovery in Database, Data Engineering Tools, High-level Query Answering, Applications in Query Optimization.

Postal Address	4-192 EE/CS Bldg., 200 Union Street S.E., Minneapolis, MN 55455, USA.
Electronic Addresses	shekhar@cs.umn.edu, hamidzad@cs.umn.edu, akohli@cs.umn.edu, coyle@cs.umn.edu
Telephone Numbers	(612) 624-8307, (612) 626-7703, (612) 625-4012
Fax	(612) 625-0572

Learning Transformation Rules for Semantic Query Optimization: A Data-Driven Approach

Shashi Shekhar, Babak Hamidzadeh, Ashim Kohli, Mark Coyle

Computer Science Dept., University of Minnesota, Minneapolis, MN 55455

1. Introduction

Semantic Query Optimization (SQO) uses query transformation rules, *e.g.* semantic integrity constraints and functional dependencies for use in query optimization. The original query is transformed into syntactically different, but *semantically equivalent* queries, which produce the same result for all database instances that satisfy the integrity constraints and functional dependencies. The objective of a semantic query optimizer at this point is to find a semantically equivalent query which yields a more efficient execution plan[1-7]. Semantic query optimization also provides the flexibility to add new information and optimization methods to an existing optimizer. A modular arrangement of optimization methods makes it possible to add, delete and modify individual methods, without affecting other methods. Since this system is rule-based it provides an extensible system for maintaining and managing optimization. Semantic query optimization is well-motivated in the literature[1, 2, 8, 9] as adding a new dimension to conventional query optimization.

The success of a semantic query optimizer depends on the availability of an effective set of query transformation rules which can reduce the execution cost of a large set of queries. The set of query transformation rules usually includes user-defined integrity constraints which are true for all possible states of a given database. User-defined rules usually form a small subset of all the query transformation rules, since in many applications it is very difficult to identify all of the relevant query transformation rules. Further, the set of query transformation rules can be expanded significantly by incrementally adding additional discovered query transformation rules based on the current state of the database[7].

We regard the unearthing of query transformation rules for semantic query optimization as a discovery task [7, 10], rather than as a concept learning task. While there may exist some general heuristics that facilitate the search for regularities, the direction of the search for such regularities is not generally prespecified. We further consider the task of unearthing query transformation rules as the discovery of qualitative laws that can be represented as logical implications.

In this paper we propose a data-driven discovery approach to learning query transformation rules. A data distribution-based approach is useful for semantic query optimization for two reasons. First, discovery of the specific patterns in the data distribution can identify useful query transformation rules. Second, it can identify the situations where the data-distribution is not uniform. These are the situations where the conventional query optimizer is less effective and where a semantic query optimizer can provide vital information to improve the query execution plans.

We utilize a grid representation of the data distribution which facilitates learning query transformation rules associated with the chosen grid.

Fig. 1 shows our proposed data-driven framework of query transformation rule discovery and query processing in a database system. A set of attributes are selected for the formation of a data distribution grid. These attributes may be specified by a database system administrator or

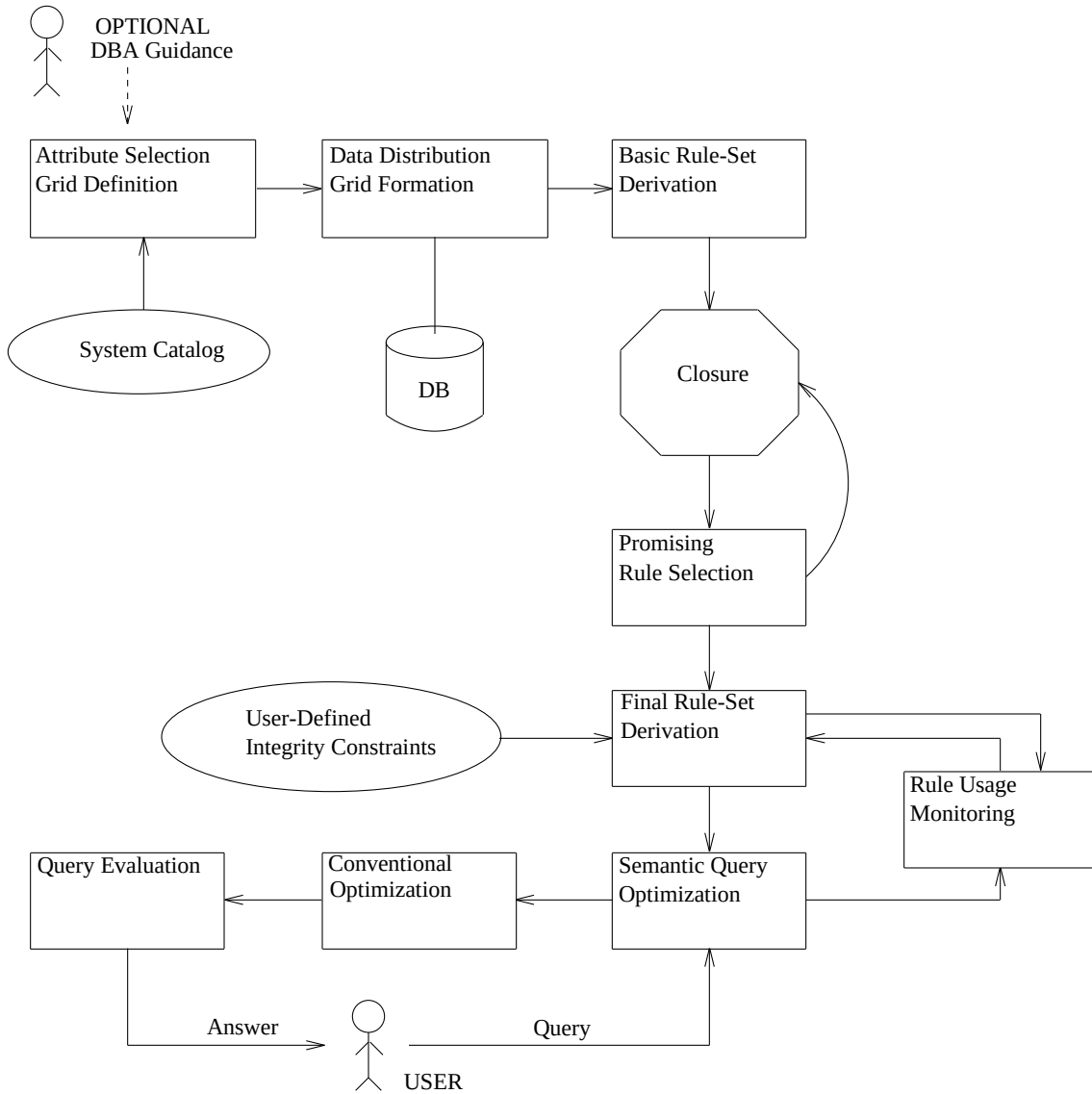


Fig. 1.: *Data-Driven Framework for Semantic Query Optimizer with Learning.*

be chosen automatically. A basic set of rules are then extracted from the grid and are supplied to a closure algorithm for the derivation of additional rules. The resulting rule-set is then pruned by a rule-selection mechanism, which discards rules with negligible cost savings. Additional rules can be deleted from the rule set by updates to the database. The resulting rule-set is then used by a semantic query optimizer to transform a user-specified query into a semantically equivalent query with a smaller execution cost. A procedural description of our discovery framework for SQO is as follows.

- 1) Select attribute set and grid coordinates.
- 2) Create a data distribution grid.

- 3) For each row, extract a set of basic rules.
- 4) Obtain the closure of the rule-set resulting from step 2 to discover other rules implied by the basic rules.
- 5) Evaluate each rule for its potential query cost savings. Discard rules with negligible cost savings.

We will use a well-known database schema[1] to illustrate various definitions and algorithms. The database schema, the relation sizes, and the various indexes available are shown in Table 1.

Relation	Attributes	Attribute for Primary Index (Clustered)	Attributes for Secondary Indices (Unclustered)	Relation Size (in tuples)
SHIPS	ShipName, Owner, ShipType, Draft, DeadWt, Capacity, Registry	ShipName	ShipType, Owner, DeadWt	20,000
PORTS	PortName, Country, Depth FacilityType	PortName	---	1,000
CARGOES	Ship, Destination, Shipper CargoType, Quantity, DollarValue, Insurance	Ship	Destination, Insurance	25,000
OWNERS	OwnerName, Location, Assets, Business	OwnerName	Business	1,000
POLICIES	Policy, Issuer, Coverage	Policy	Issuer	25,000
INSURERS	Insurer, InsCountry, Capitalization	Insurer	---	500

Table 1.: The Shipping Database:(System Catalog).

The implicit joins that underlie the logical access paths among the relations are listed in Fig. 2.

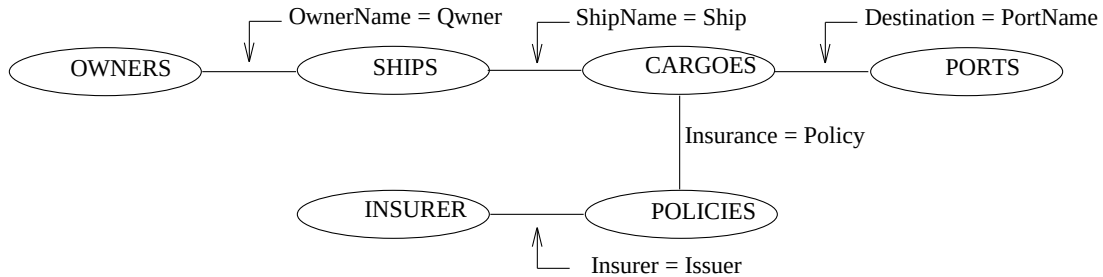


Fig. 2.:Unique Join Predicates.

The remainder of this paper is organized as follows: Section 2 investigates the related approaches to the problem of learning query transformation rules to bring out our contributions. Section 3 defines the concept of data-distribution and its relationship with query transformation rules. Section 4 presents the rule closure algorithm. Section 5 describes an extended example to illustrate the concepts. A summary of the results and conclusions are provided in section 6.

2. Related Approaches and Our Contributions

The learning of query transformation rules can be query-driven or data-driven. In query-driven frameworks [7, 11] the search for new query transformation rules is guided by the set of queries arriving at the database using query comparisons [7] and hypothesis generation and testing [11, 12]. In query comparison, the set of queries arriving after the last update are analyzed by comparing the set of tuples retrieved to answer various queries. If the set of tuples retrieved by two queries are identical then query transformation rules relating the restrictions in the two queries can be added [7]. In hypothesis generation and testing approaches [13], the queries are used to generate a candidate query transformation rule. A candidate rule consists of an antecedent restriction and the set of variables in the consequent. The antecedent restriction is generated from the restriction clauses of the queries arriving at the system. The set of free variables for the consequent is generated by heuristics such as index introduction [1]. The antecedent restriction is evaluated against the database state to retrieve and summarize all possible values of the free variables to form the consequent.

In the data-driven framework the search is directed by the data, leading to the learning of rules characterizing patterns in the data representing query transformation rules. This learning can take place off-line in order to limit the effect of larger processing times on on-line processing. An advantage of this framework for learning query transformation rules is that the rules are learned a priori and they cover many queries that may not have been requested before. Data-driven approaches can be based on the learning algorithms developed in Artificial Intelligence (AI). Many of these learning algorithms discover rules represented in languages similar to first order predicate logic (FOPL), which can be used to represent general integrity constraints [14, 15] and query transformation rules. For example, the representation languages used in AQ15 [16] and in the conceptual clustering algorithm Cluster 2 [17] are fairly close to FOPL.

The AI learning algorithms can be classified into two categories, supervised concept learning and unsupervised discovery. The supervised concept learning algorithms use an external tutor and a set of training examples with known class labels to learn a set of predetermined concepts. After training, these algorithms will be able to correctly classify a sample data into one of the learned concepts. Examples of supervised concept learning algorithms include AQ [16] and ID3 [18]. Supervised concept learning algorithms can not directly be applied to the problem of learning query transformation rules. In this domain there may be no a priori concepts to provide the set of training examples.

Unsupervised discovery is concerned with learning without the help of external tutors or examples. Previous work on algorithms for unsupervised discovery can be divided into two subclasses [19] (a) Taxonomy formation and clustering, and (b) Discovery of empirical laws.

Taxonomy formation is concerned with the discovery of classification rules. Two approaches to taxonomy formation include numerical clustering and conceptual clustering. Comprehensive surveys on clustering methods can be found in [20-24]. Numerical clustering methods partition data into clusters based on the distance in n-dimensional space. Numeric

clustering methods are not interesting since they do not generate FOPL formulas needed to represent query transformation rules. Conceptual clustering arranges data into partitions based on certain conceptual classes. The basic theory and an algorithm for conceptual clustering have been developed in [25]. Other conceptual clustering algorithms include Discon[26], RUMM-AGE [27], and Cluster 2 [17]. Conceptual clustering uses background knowledge about the functionality of clusters to guide the learning process towards the formation of more useful clusters [17, 28].

The discovery techniques to learn empirical laws can be divided into two categories, namely quantitative and qualitative law discovery. Quantitative discovery techniques use numeric-valued variables to discover mathematical functions that summarize the data. These techniques include regression analysis[29] and other function finding methods. Previous work in automatic discovery of quantitative rules includes the Bacon system [13]. Discovering quantitative rules in databases has also been pursued by a system known as the Forty-Niner [10]. This system examines several possible subsets of data for possible empirical laws. The subsets are created by using projection, slicing and aggregation operations in a multi-dimensional attribute space. The quantitative approaches cannot directly be applied to the domain of integrity constraint learning as they are not able to produce FOPL rules.

Qualitative law discovery refers to techniques which find logical relationships among the data. Such relationships could consist of finding bounds on attribute values or consist of qualitative relationships among attributes, such as $X > Y$, where X and Y are attributes. Some of the algorithms for qualitative rule discovery include AM [30] and Glauber[31]. The Glauber system[31] differs from AM[30] in that as it begins with very little domain knowledge. Both methods, however, use domain heuristics to direct the search for interesting concepts.

Frameworks for learning qualitative empirical laws are closest to the tasks of discovering query transformation rules from data. Integrity constraints represent regularities in the data distribution, which can be represented in FOPL like languages. Unfortunately, the previously designed learning algorithms like AM[30] cannot be used directly for learning query transformation rules due to their dependence on the domains for which they were developed. Most of qualitative empirical law discovery systems focus on learning number-theoretic, algebraic, statistical or qualitative physics oriented formulas. These techniques are not suitable for learning FOPL formulas typical of query transformation rules.

Contributions: We propose an empirical law discovery system for the problem of query transformation rule learning. Our approach is data-driven in contrast to the previous approaches [7, 11] to learning query transformation rules. Being query driven, previous approaches have the disadvantage that cost savings occur only if queries are repeated. This approach might incur larger costs in the case where many new queries arrive. Also, due to the lack of relevant queries, many useful query transformation rules may never be learned. We formalize the notion of data distributions by defining concepts of grids, coordinates, and range. We characterize the relationship between data distributions and query transformations. We clarify the notion of query equivalence, rule correctness and rule equivalence with respect to a data distribution. We discuss the domain knowledge needed about the attributes in a database to learn query transformation rules. We also provide a data-driven algorithm to learn query transformation rules from the data distributions. We provide closure rules based on the domain knowledge and notion of rule correctness to derive new rules from the basic rules. We provide correctness, completeness and complexity analysis of the algorithms and illustrate their usefulness via a sequence of examples.

3. Learning Transformations Rules for Semantic Query Optimization

In this section, we first define the representation language for queries, integrity constraints and transformation rules. We then characterize the class of transformation rules which should be learned for semantic query optimization. We then show the correspondence between basic transformation rules and patterns in the data-distribution, which forms the basis of the data-driven rule discovery algorithm.

3.1. Representation Language

We follow a logic-based representation proposed in [1, 14, 15] for queries, integrity constraints and query transformation rules in this paper. For relations P , the atomic formula will be written as $P(a_1 \text{ op } t_1, \dots, a_n \text{ op } t_n)$, where $a_1, \dots, a_n$ are some attributes of P . The operation 'op' is a comparison operator which will include $=$, $<$, and $>$. Well-formed formulas can be constructed from atomic formulas by using unary quantifiers (*there exists*), (*for all*), and connectives 'not', 'or', 'and', ' $\rightarrow$ '. Well-formed formulas can be used to represent integrity constraints. The formulas with some attributes marked by the symbol '?' can represent queries. We will omit the names of the predicates without loss of precision since attribute names are unique. We will also omit (*for all*) quantifiers by implying universal quantification of free variables. The formal rules for dropping the quantifiers are discussed in [1]. Furthermore, we will assume the queries to be defined on a natural join of all relations in the shipping database.

Example: Query = Find the destinations of all ships with DeadWt $>$ 400 and DollarValue $>$ 4000.

Can be represented as:

$$\begin{aligned} Q_0 &= \text{SHIPS}(\text{DeadWt} > 400) \text{ and } \text{CARGOES}(\text{DollarValue} > 4000, ? \text{ Destination}) \\ &= (\text{DeadWt} > 400) \text{ and } (\text{DollarValue} > 4000) : (? \text{ Destination}) \end{aligned}$$

Example: Integrity Constraint/Query Transformation Rule IC_0 :

Any Ship of type SuperTanker with a cargo of DollarValue $>$ 4000 is insured by Lloyds.
 IC_0 : for all x [(Ships(ShipName = x , ShipType = SuperTanker) and
 Cargo(Ship = x , DollarValue $>$ 4000) $\rightarrow$ Policies(Issuer = 'Lloyds'))]
 IC_0 : = (ShipType = 'SuperTanker') and (DollarValue $>$ 4000) $\rightarrow$ (Issuer = 'Lloyds')

Example: Integrity Constraint/Query Transformation Rule IC_1 :

There is at least one ship of type Tanker insured by Lloyds.
 IC_1 : *there exists* x (Ships(ShipName = x , ShipType = 'tanker') $\rightarrow$ Policies (Issuer = 'Lloyds'))

The integrity constraint (IC_1) cannot be simplified by dropping the existential quantifier, since the resulting rule (ShipType = 'Tanker') $\rightarrow$ (Issuer = 'Lloyds') will not be correct for all tuples of the database.

We limit our attention to universally quantified conjunct queries such as Q_0 , where the variable marked with '?' is universally quantified, and quantification clauses are connected by conjunction. The answer to this query includes all possible instances of the marked variable satisfying the formula. Universally quantified queries are expensive to execute and are the prime candidates for optimization. Non-conjunct queries can be represented as disjunctions of conjunct sub-queries, which can be optimized using the proposed framework. We also attach an alternate

relational interpretation to query Q as $R_x(Q)$, representing a relation containing the set of tuples retrieved by query in a current database state x [7].

Definition: Two queries Q_1 and Q_2 are semantically equivalent in the current database state x if $R_x(Q_1) = R_x(Q_2)$.

We note that semantically equivalent queries Q and Q_1 can be considered to be logically equivalent expressions in the current state of the database, since their restrictions are logically equivalent in the current state of the database.

3.2. Integrity Constraints and Query Transformation Rules

Integrity constraints on a relational database define the intended semantics of its relations[32-37]. Most databases have certain integrity constraints on the data that must be enforced while manipulating the database. A simple type of integrity constraints is the data type and a range of valid values that is specified for each data item. Another integrity constraint may impose uniqueness constraints on data items. Other integrity constraints might specify relationships between records in different files.

Some constraints are implicitly represented in the data model. An example of implicit constraints in the relational model is functional dependency. Other constraints might be inherent in the data model. These constraints would be assumed to hold by the data model definition, without being specified in a schema. An example of an inherent constraint in the relational model is that values in the data base are atomic and indivisible. In addition to implicit and inherent constraints, there are other constraints on the data that have to be explicitly specified. An example of an explicit constraint can be the range of valid values for an attribute or more complex relationships between attribute values. The constraint on employee salaries which states that an employee's salary can never be higher than his manager's salary is an instance of such constraints.

Another classification of integrity constraint types is based on the current and historic states of a data base. State constraints refer to all constraints on the data that must hold on each state of the database[7]. Transition constraints constrain the way in which one state of the database can be transformed to another state. A rule ensuring monotonically increasing salary values is a transition constraint[32, 34-36].

We distinguish between integrity constraints and query transformation rules to provide a more accurate description of their nature. Query transformation rules are explicit state integrity constraints, which are useful for semantic query optimization for universally quantified queries. These rules are characterized by universal quantification in the well-formed formulas representing those queries. For example, IC_0 is a query transformation rule but IC_1 is not a query transformation rule. Query transformation rules subsume the "simple rules" learned in rule discovery for query optimization [12], since "simple rules" are universally quantified. These rules also subsume the rules learned via automatic knowledge acquisition[7], since the latter are based on set comparisons. Query transformation rules represent the useful integrity constraints which can provide savings during semantic query optimization.

3.3. User-Defined and Discovered Query Transformation Rules

User-defined transformation rules are a subset of user-defined explicit integrity constraints. Since a database state provides a set of tuples for each one of the considered relations, the user-defined query transformations hold true for the set of tuples in each valid state of the database. Any updates violating a user-defined transformation rule is dis-allowed.

Discovered transformation rules from a database state may or may not represent a time-independent property of the database. For example, in the shipping database, all ships of the type "Tanker" may be insured by "Lloyds" in a specific state of the database possibly because there is only one ship of the type "Tanker" in that particular state. Hence we can discover and make use of the query transformation rule:

$IC_2: (ShipType = "Tanker") \rightarrow (Insurer = "Lloyd") \text{ and } (ShipName = "a")$

Any queries on the tanker class could be answered by examining the tuple with ShipName = "a" in the current state of the database. Finding such query transformation rules will be beneficial to the semantic query optimizer. A discovered transformation rule may lead to a user-defined integrity constraint. In general, a discovered query transformation rule may be invalidated by future updates to the database.

Definition: A user-defined query transformation rule is a universally quantified explicit-state integrity constraint, which is true in all valid database states. A discovered query transformation rule is a universally quantified well-formed formula which is true in the current database state.

A query transformation rule is considered correct if it can be used for semantic query optimization in the current state of the database. A correct query transformation rule holds in the current state of the database, and it transforms a given query to a semantically equivalent query. User-defined query transformation rules are correct, since they hold on all database states. However, discovered query transformation rules may not be correct in all database states.

During semantic query optimization, a query Q can be transformed into $(Q \text{ and } (A \rightarrow C))$. If Q is a conjunctive query and it has a clause A , then the query Q and $(Q \text{ and } C)$ are equivalent if $A \rightarrow C$, because of the logical equivalence of $(A \text{ and } (A \rightarrow C))$ and $(A \text{ and } C)$. Thus, the correctness of a query transformation rule $A \rightarrow C$ can be determined from the equivalence between the logical query expressions A and $(A \text{ and } C)$. A query transformation rule $A \rightarrow C$ is correct in the current state of a database if the logical queries A and $(A \text{ and } C)$ are equivalent queries in the current state of the database.

A query transformation rule is useful for a query Q if it helps in substantially reducing the execution cost of query Q during semantic query optimization. For example, the query transformation rule $(ShipType = "tanker") \rightarrow (ShipName = "a")$ can be used to transform query $Q = (ShipType = "tanker") : (? ShipName)$ to $Q' = (ShipName = "a") : (? ShipName)$. The transformed query Q' can be answered with no data access, resulting in saving the cost of scanning the SHIPS relation, to answer Q .

Definition 3: A query transformation rule (QTR) is useful for query Q if $f(Q, Q') > t$, where Q' is the query generated from Q by applying QTR, and f is a function representing cost savings in transforming Q to Q' , and t is a threshold.

Trivial rules of the kind $(A \text{ and } C) \rightarrow A$ are correct but not useful for semantic query optimization since these cannot provide any cost savings.

4. Data Distribution-Based learning

In this section, we first define the notion of data-distributions and show their relationship to query transformation rules. We then discuss general issues in data-driven learning of query transformation rules for a grid-based data-distribution.

4.1. Data Distributions and Domain Knowledge

A data distribution is a formal representation of a state of a given database. A data distribution is a function which maps a grid and a database to the number of tuples in each cell of the grid. A grid is a regular decomposition of the space of all possible tuples in the database into a collection of cells. Cells are disjoint and together encompass the entire space of the possible tuples in the database, for the coordinates of the grid.

A grid is specified by a set of coordinates, which are functions of the attributes in the database. The dimensionality of a grid represents the number of coordinates it has, assuming that the coordinates are linearly independent. A grid is orthogonal to the attributes in a database if each coordinate axis represents a unique attribute in the database. A grid is oblique if a coordinate represents a function of two or more attributes. Example grids for the shipping database include $\langle \text{Capacity}, \text{Quantity} \rangle$ and $\langle \text{Capacity} + \text{Quantity}, \text{Capacity} - \text{Quantity} \rangle$. The former is an orthogonal grid with respect to the attributes in the shipping database and the latter is an oblique grid.

The cells of a grid are formed by dividing the values possible for each coordinate into a finite set of ranges. Let S_i ($i = 1 \dots n$) denote the set of ranges for the n coordinates of an n -dimensional grid. The set of cells for this grid can be represented by the cross-product of its range-sets, i.e. $S_1 \times S_2 \times \dots \times S_n$.

Example: Let us consider a grid (see Fig. 3) with a coordinate-set = $\langle \text{BusinessType}, \text{CargoType} \rangle$. The range-sets for the first coordinate may be (Petroleum, Non-Petroleum). The range-set for the second coordinate may be (NaturalGas, RefinedOil, Others). The cells of this grid are (Petroleum, NaturalGas), (Petroleum, RefinedOil), (Petroleum, Others) (Non-Petroleum, NaturalGas), (Non-Petroleum, RefinedOil), and (Non-Petroleum, Others).

	BusinessType		
Petroleum	1230	300	0
Non-Petroleum	356	30	523
	NaturalGas	RefinedOil	Others
	CargoType		

Fig. 3.: Data distribution in the grid $\langle \text{CargoType}, \text{BusinessType} \rangle$.

A data distribution of tuples in the shipping database may be depicted by placing integers in each cell representing the number of tuples falling into each cell, or qualifying the selection query corresponding to the various cells, as is shown in *Fig. 3*.

We distinguish between basic coordinate-ranges, aggregate coordinate-ranges, cross-product ranges and cells. The basic coordinate-ranges are the smallest atomic ranges associated with individual coordinates of a grid. For example, the ranges (NaturalGas, RefinedOil, Others, Petroleum, Non-Petroleum) are basic coordinate ranges in *Fig. 3*. The aggregate coordinate ranges are collections of basic coordinate ranges associated with a unique coordinate of a grid. The ranges ((NaturalGas, RefinedOil), (Petroleum, Non-Petroleum) and (NaturalGas, RefinedOil, Others)) in *Fig. 3* are examples of aggregate coordinate ranges. The cross-product ranges are formed as conjunction of the coordinate-ranges, on different coordinates. Some examples are (NaturalGas, Petroleum), and ((NaturalGas, RefinedOil), Petroleum) in *Fig. 3*. Cells are special classes of basic cross-product ranges involving conjunction of a basic coordinate-range for all coordinates in the grid. Thus, cells can be considered as atomic ranges in the grid. All ranges can be represented as a collection of cells. For a grid with C cells there are at most 2^C distinct ranges defined over the grid. Finally, a range is called minimal with respect to a data-distribution, if it does not contain any cells with zero tuples. Ranges can be viewed as logical expressions which are true for all tuples contained in themselves.

The operations on ranges include union, intersection and negation. The definition of operations depend upon the nature of the ranges as well as the nature of the underlying attribute. The dependence on the nature of range defined as follows:

$$\begin{aligned} \text{union}(R1, R2) &= \begin{cases} \text{interval-union}(R1, R2) & \text{if } R1, R2 \text{ are coordinate-ranges on a common coordinate} \\ \text{logical-disjunction}(R1, R2) & \text{otherwise} \end{cases} \\ \text{intersection}(R1, R2) &= \begin{cases} \text{interval-intersection}(R1, R2) & \text{if } R1, R2 \text{ are coordinate-ranges on a common coordinate} \\ \text{logical-conjunction}(R1, R2) & \text{otherwise} \end{cases} \\ \text{negation}(R) &= \begin{cases} \text{interval-negation}(R) & \text{if } R \text{ is a coordinate-range} \\ \text{logical-negation}(R) & \text{otherwise} \end{cases} \end{aligned}$$

The logical-disjunction, logical-conjunction and logical-negation operations represent the 'or', 'and', and 'not' operations, respectively. The interval-operations are discussed with respect to the nature of the underlying attributes, next.

The domain of attributes in a database can be divided into two classes: the ordered- and the unordered-valued attributes [7]. Unordered attributes are those whose values do not belong to an ordered domain. For example, there is no ordering relationships between the values of the Color attribute (e.g. red, blue, green). The values a and b of unordered attributes A and B can only be compared via $\{=, \neq\}$ comparison operators, namely $a=b$ or $a \neq b$.

Ordered attributes are those whose values do belong to an ordered domain. The domain of ordered attributes can be divided into discrete- and continuous-valued attributes. Discrete-valued attributes are those whose values are integer-valued. Continuous-valued attributes are those whose values are real-valued. The values of ordered attributes can be compared via $\{<, \leq, >, \geq, =, \neq\}$ comparison operators. Next, we define interval-union, interval-intersection and interval-negation operations for ordered and unordered attributes that are used to combine, connect or modify one or more restrictions on attributes.

In the case of an unordered attribute A , an interval-negation operation $\text{Not}(A)$ corresponds to the set-theoretic complement operation. Thus, if the domain of attribute A is the set D , then $\text{Not}(A \in B) = (A \in C)$ such that $D - B = C$, where B and C are subsets of D . The negation cases for an ordered attribute A are enumerated in *Table 2*.

A	$\text{Not}(A)$
$a=x$	$(a>x) \text{ OR } (a<x)$
$a>x$	$a \leq x$
$a<x$	$a \geq x$

Table 2: $\text{Not}(A)$ operator for intervals.

In the case of ordered attributes $A1$ and $A2$, an interval-union operation, $\text{Union}(A1, A2)$, corresponds to the set-theoretic union operation. The different union cases for ordered attributes $A1$ and $A2$ are enumerated in *Table 3*.

Clauses	$X = Y$	$X > Y$	$X < Y$
$A1:(a=x), A2:(a=y)$	$a=x$	$(a=x) \text{ or } (a=y)$	$(a=x) \text{ or } (a \leq y)$
$A1:(a>x), A2:(a=y)$	$a \geq x$	$(a>x) \text{ or } (a \leq y)$	$a>x$
$A1:(a<x), A2:(a=y)$	$a \leq x$	$a<x$	$(a<x) \text{ or } (a \leq y)$
$A1:(a=x), A2:(a>y)$	$a \geq x$	$a>y$	$(a=x) \text{ or } (a>y)$
$A1:(a=x), A2:(a<y)$	$a \leq x$	$(a=x) \text{ or } (a<y)$	$a<y$
$A1:(a>x), A2:(a>y)$	$a>x$	$a>y$	$a>x$
$A1:(a>x), A2:(a<y)$	$(a \neq x) \text{ or } (a<x) \text{ or } (a>x)$	$(a>x) \text{ or } (a<y)$	$-\infty < a < +\infty$
$A1:(a<x), A2:(a<y)$	$a<x$	$a<x$	$a<y$
$A1:(a<x), A2:(a>y)$	$(a \neq x) \text{ or } (a<x) \text{ or } (a>x)$	$-\infty < a < +\infty$	$(a<x) \text{ or } (a>y)$

Table 3: $\text{Union}(A1, A2)$ operator for clauses on common attribute a .

In the case of unordered attributes $A1$ and $A2$, an interval-intersection operation, $\text{Intersection}(A1, A2)$, corresponds to the set-theoretic intersection operation. The different intersection cases for ordered attributes are enumerated in *Table 4*. Note that the intersection of restrictions on different attributes are represented as a conjunction of the two restrictions connected by a logical AND operator meaning that both of the restrictions have to be satisfied for the intersection to hold true.

We define range-closure $\text{RC}(S)$ of a set of ranges S with respect to the range operations recursively by the following rules:

- 1) Ranges in S belong to the range-closure of S , $\text{RC}(S)$.
- 2) If $r1 \in S$, $r2 \in S$, then $\text{union}(r1, r2)$, $\text{intersection}(r1, r2)$, $\text{negation}(r1)$ and $\text{negation}(r2)$ are in $\text{RC}(S)$.

We note that the range closure of a set of ranges from a finite grid is also finite. For a 2-dimensional grid with k basic coordinate-ranges per coordinate, the number of possible ranges is limited by the number of subsets of the K^2 basic cross-product ranges. We further note that the range-closure of a set of minimal ranges results in another collection of minimal ranges.

Clauses	$X = Y$	$X > Y$	$X < Y$
A1:(a=x), A2:(a=y)	a=x	NULL	NULL
A1:(a>x), A2:(a=y)	NULL	NULL	a=y
A1:(a<x), A2:(a=y)	NULL	a=y	NULL
A1:(a=x), A2:(a>y)	NULL	a=x	NULL
A1:(a=x), A2:(a<y)	NULL	NULL	a=x
A1:(a>x), A2:(a>y)	a>x	a>x	a>y
A1:(a>x), A2:(a<y)	NULL	NULL	x<a<y
A1:(a<x), A2:(a<y)	a<x	a<y	a<x
A1:(a<x), A2:(a>y)	NULL	y<a<x	NULL

Table 4: *Intersection(A1,A2) operator for ranges on common attribute a.*

4.2. Discovering Basic Query Transformation Rules

Integrity Constraints (and user-defined query transformation rules) give rise to special data-distribution patterns in the grid formed by the set of attributes involved in the integrity constraint. For example, the integrity constraint (Business = 'Petroleum') $\rightarrow$ (CargoType $\in$ {'Natural Gas', 'Refined Oil'}) on the shipping database results in a data-distribution with no tuples in the cell ('Petroleum', 'Others'), as is shown in Figure 3. In general, an integrity constraint of the type (Attribute1 = 'a') $\rightarrow$ (Attribute2 = 'b') leads to no tuples in cells satisfying ((Attribute1 = 'a') and (Attribute2 $\neq$ 'b')). Furthermore, the data-distribution patterns formed by integrity constraints and user-defined query-transformation rules are time invariant.

Patterns in a data-distribution for the current state of the database, which are not implied by user-defined query transformation rules can be used to find "discovered" query transformation rules. A set of simple patterns are shown in Table 5 along with the query transformation rules which can be discovered on the basis of these patterns.

Data Distribution Pattern	Universally Quantified Query Transformation Rules
Column (A1 = 'a') has all tuples in the cell corresponding to (A2 = 'b') in orthogonal grid <A1,A2>	(A1 = 'a') $\rightarrow$ (A2 = 'b')
Column (A1 = 'a') has all tuples in the cells corresponding to (A2 = 'b') or (A2 = 'c')	(A1 = 'a') $\rightarrow$ (A2 $\in$ {b,c})
Column ((A3 + A4) = 'e') has no tuples in oblique grid <(A3 + A4),(A3 - A4)>	(A3 + A4 $\neq$ 'e')
Column ((A1 = 'a') and (A2 = 'b')) has all tuples in the cell corresponding to (A3 $\neq$ 'c') in the grid <A1,A2,A3>	(A1 = 'a') and (A2 = 'b') $\rightarrow$ (A3 $\neq$ 'c')

Table 5.: *Equivalent Simple Rules and Data Distribution Patterns*

Query transformation rule-set S can be characterized with respect to data distributions as a triplet $\langle AS, CS, \text{Semantics} \rangle$. AS is the antecedent set of ranges defined over the grid. CS is the consequent set of ranges defined over the grid. Individual rules $A \rightarrow C$ in S must satisfy $A \in AS$ and $C \in CS$ and a specified rule semantics with respect to the data distribution. We note that the antecedent set AS and consequent set CS can have at most 2^C (where C is the number of cells in the grid) logically distinct members, even though they may have more syntactically distinct members. Rules can have several alternative semantics including the ones shown in Table 6.

Semantics Class	Rule	Semantics
Universal	$A \rightarrow C$	All tuples in range A are also in range C in the data distribution.
Existential	$A \rightarrow C$	Some tuples in range A are also in range C , in the data distribution.
Statistical	$A \rightarrow C:K$	$\Pr.(t \in C \mid t \in A) = k$ in the data distribution.

Table 6. Rule Semantics.

We will focus on the universal semantics for the rules, and represent rule sets by $\langle AS, CS \rangle$. Furthermore, we will restrict our attention to $AS = \{\text{Set of coordinate-ranges}\}$, since this represents the set of interesting query transformation rules for semantic query optimization.

Definition: A query transformation rule $A \rightarrow C$ is correct with respect to a data distribution, if any tuple in range A is also in range C , i.e. A and $(A \text{ and } C)$ are equivalent queries.

Definition: A query transformation rule $A \rightarrow C$ is symmetric with respect to a data-distribution if $C \rightarrow A$ is a correct query transformation in the data distribution, i.e. $R_x(A) = R_x(C)$. We note that the notion of symmetric rules excludes some of the useful query transformations. We further note that symmetric rules of the kind $A \rightarrow C$ are such that at least one of $(A \rightarrow C, C \rightarrow A)$ will be a useful rule.

Definition: Rule Equivalence: Query transformation rules $A1 \rightarrow C1$ and $A2 \rightarrow C2$ are equivalent in a data-distribution D , if:

- 1) $A1$ and $A2$ are equivalent queries and
- 2) $C1$ and $C2$ are equivalent queries on D .

Symmetric rules $A \rightarrow C$ and $A' \rightarrow C'$ are equivalent if $R_x(A) = R_x(A')$ or $R_x(C) = R_x(C')$. We note that rule-equivalence is different from logical equivalence, since all correct rules are logically equivalent, true expressions on a data-distribution. Instead, rule equivalence captures the equivalence of rules with respect to their usefulness in semantic query optimization. Equivalent useful rules $A \rightarrow C$ and $A \rightarrow C'$ will be leading to similar cost savings if C and C' are defined on the same set of attributes on a common grid.

Definition: A rule-set $DRS = \langle DAS, DCS \rangle$ can be derived from a given rule-set $RS = \langle AS, CS \rangle$ for a data distribution, such that $DAS = \text{range-closure}(AS)$, $DCS = \text{range-closure}(CS)$ and rules in DRS obey (a) the universal semantics with respect to the data distribution, and (b) they are symmetric rules.

It can now be formally shown that the query transformation rules that are implied from the data-distribution patterns are correct rules for semantic query optimization in the current state of the database. We will consider three basic rules for showing the correctness. More complex rules can be derived from the basic rules via a closure algorithm described in the next section.

The basic data-distribution patterns (DDP) on a current database state and simple query transformation rules (QTR) are discussed now. In the following propositions, symbols in single quotes represent basic coordinate-ranges. We will describe a way of learning minimal-consequent query transformation rules from some patterns in a given data distribution.

DDP1: Column $(A1 = 'a')$ has tuples only in the cells corresponding to $(A2 = 'b_1')$, $(A2 = 'b_2')$, ..., and $(A2 = 'b_k')$ in a 2-dimensional, orthogonal grid $\langle A1, A2 \rangle$.

QTR1: $(A1 = 'a') \rightarrow (((A2 = 'b_1') \text{ OR } (A2 = 'b_2') \text{ OR } \dots \text{ OR } (A2 = 'b_k')) \text{ and } (A1 = 'a'))$

Proposition 1: If the current state of the database has a data distribution pattern DDP1 then query transformation rule QTR1 is correct and symmetric.

Proof: We note that the QTR is symmetric since its consequent implies its antecedent. The antecedent $A = (A1 = 'a')$ and conjunction of antecedent and consequent A and $C = (A1 = 'a')$ and $\{(A2 = 'b_1') \text{ OR } (A2 = 'b_2') \text{ OR } \dots \text{ OR } (A2 = 'b_k')\}$ are semantically equivalent in the current state of the database. This can be verified by constructing the relations $R_x(A)$ and $R_x(C)$ representing the answers to queries A and A and C . The relations $R_x(A)$ and $R_x(A \text{ and } C)$ are equal because of the existence of the data distribution pattern DDP1. This can be shown by contradiction. Let us assume that there is a tuple $t \in R_x(A)$, which is not a member of $R_x(A \text{ and } C)$. In this case column $(A1 = 'a')$ in the data-distribution grid $(A1, A2)$ will have a tuple outside the cells satisfying A and C . This leads to the contradiction with the existence of data-distribution pattern DDP1. Hence the queries A and $(A \text{ and } C)$ are equivalent.

DDP2: Column $C = f_1(A1, A2) = 'a'$ has tuples only in the cells corresponding to $(f_2(A1, A2) = 'b_1')$, ..., $(f_2(A1, A2) = 'b_k')$ in the oblique grid $\langle f_1(A1, A2), f_2(A1, A2) \rangle$.

QTR2: $(f_1(A1, A2) = 'a') \rightarrow (((f_2(A1, A2)) \in [b_1, b_2, \dots, b_k]) \text{ and } (f_1(A1, A2) = 'a'))$

Proposition 2: If the current state of the database has a data distribution pattern DDP2 then query transformation rule QTR2 is correct and symmetric.

Proof: The antecedent $A = (f_1(A1, A2) = 'a')$ and conjunction of antecedent and consequent A and $C = (f_1(A1, A2) = 'a')$ and $\{(f_2(A1, A2)) \in [b_1, b_2, \dots, b_k]\}$ are semantically equivalent in the current state of the database. This can be verified by constructing the relations $R_x(A)$ and $R_x(C)$ representing the answers to queries A and $(A \text{ and } C)$. The relations $R_x(A)$ and $R_x(A \text{ and } C)$ are equal because of the existence of the data distribution pattern DDP2. This can be shown by contradiction. Let us assume that there is a tuple $t \in R_x(A)$, which is not a member of $R_x(A \text{ and } C)$. In this case column $(f_1(A1, A2) = 'a')$ in the data-distribution grid $\langle f_1(A1, A2), f_2(A1, A2) \rangle$ will have a tuple outside the cells satisfying A and C . This leads to the contradiction with the existence of data-distribution pattern DDP2. Hence the queries A and $(A \text{ and } C)$ are equivalent.

DDP3: Column $C = ((A1 = 'a') \text{ and } (A2 = 'b') \text{ and } \dots \text{ and } (A(k-1) = 'j'))$ has tuples only in the cells corresponding to $(A1 = 'k_1')$, $(A1 = 'k_2')$, ..., $(A1 = 'k_n')$ in a K-dimensional orthogonal grid $\langle A1, A2, \dots, A(k-1), A_k \rangle$.

QTR3: $(A1 = 'a') \text{ and } (A2 = 'b') \text{ and } \dots \text{ and } (A(k-1) = 'j') \rightarrow ((A_k \in ['k_1', 'k_2', \dots, 'k_n']) \text{ and } ((A1 = 'a') \text{ and } (A2 = 'b') \text{ and } \dots \text{ and } (A(k-1) = 'j')))$

Proposition 3: If the current state of the database has a data distribution pattern DDP3 then query transformation rule QTR3 is correct and symmetric.

Proof: The antecedent $A = (A1 = 'a')$ and $(A2 = 'b')$ and ... and $(A(k-1) = 'j')$ and conjunction of antecedent and consequent A and $C = \{(A1 = 'a')$ and $(A2 = 'b')$ and ... and $(A(k-1) = 'j')\}$ and $\{(A_k \in ['k_1', 'k_2', \dots, 'k_n'])\}$ are semantically equivalent in the current state of the database. This can be verified by constructing the relations $R_x(A)$ and $R_x(C)$ representing the answers to queries A and A and C . The relations $R_x(A)$ and $R_x(A \text{ and } C)$ are equal because of the existence of the data distribution pattern DDP3. This can be shown by contradiction. Let us assume that there is a tuple $t \in R_x(A)$, which is not a member of $R_x(A \text{ and } C)$. In this case column $(A1 = 'a')$ and $(A2 = 'b')$ and ... and $(A(k-1) = 'j')$ in the data-distribution grid $(A1, A2, \dots, A_k)$ will have a tuple outside the cells satisfying A and C . This leads to the contradiction with the existence of data-distribution pattern DDP3. Hence the queries A and $(A \text{ and } C)$ are equivalent.

Definition: Basic rule-set RS for a grid is defined by the triplet $\langle AS, CS, \text{Universal Semantics} \rangle$, where AS is equal to $AS = \{r \mid r \text{ is a basic coordinate-range on a unique coordinate of the grid}\}$, and $CS = \{r \mid r \text{ is a range in the grid}\}$, and the rules are correct and symmetric.

We note that Proposition 1, 2 and 3 can be used to derive symmetric basic rule-sets for different grids.

Example: According to the grid of *Fig. 3*,

$AS = \{\text{Petroleum, Non-Petroleum}\}$
 $CS = \{\text{NaturalGas, RefinedOil, Others, (NaturalGas, RefinedOil),}$
 $(\text{NaturalGas, RefinedOil, Others}), (\text{RefinedOil, Others}), (\text{NaturalGas, Others})\}$

Some example rules in the basic rule-set of this grid are as follows:

$(\text{BusinessType} = \text{'Petroleum'}) \rightarrow ((\text{CargoType} \in \{\text{'NaturalGas'}\}) \text{ and Antecedent})$
 $(\text{BusinessType} = \text{'Petroleum'}) \rightarrow ((\text{CargoType} \in \{\text{'NaturalGas'}, \text{'RefinedOil'}\}) \text{ and Antecedent})$
 $(\text{BusinessType} = \text{'Non-Petroleum'}) \rightarrow ((\text{CargoType} \in \{\text{'NaturalGas'}, \text{'RefinedOil'}, \text{'Others'}\}) \text{ and Antecedent})$

4.3. Choice of Data Distribution Grids

There are several decisions about the construction of the data distribution grid that can affect performance in terms of the type of rules that can be learned, and the speed of learning. One such decision is the choice of the grid. Examples of possible grid structures are euclidean grids and polar grids. In euclidean grids, the axes along which the attributes are represented are aligned with the $x, y, z, \dots$ axes of a euclidean grid. Variations of this type of grid may allow rotational transformations of the grid around an origin to allow better classification of the data. In polar grids, the partitions are formed by circles that are at different distances from an origin, and by radii across the circles, originating at the origin. The choice of the grid structure depends on the way the data is structured and on the representation which lends itself to better classification of the data.

Another characteristic of the grid is the set of attributes used to define the coordinates. The number of such attributes decide the dimensionality of the grid. As will be discussed later, more useful rules can be learned from grids representing 2 or 3 attributes. Also, in higher dimensions, rules become more complex (i.e. larger number of attributes and disjuncts in the antecedent) which apply to fewer queries. their limited applicability to queries reduces their usefulness to

semantic query optimization. The choice of attributes is a major issue in constructing grids. The attributes and their relationships decide the types of rules that can be learned from the grid. Several strategies can be used in selecting the attribute set. One such strategy is to allow the user to provide a heuristic[1] as to what attributes to choose. The user can use his domain knowledge, and his knowledge of attribute spaces to select a set of attributes to be represented in the grid.

Another strategy in attribute selection is the use of the information in the database system catalog. The database catalog can provide useful information which can be used in selecting the attributes for the grid. One such information is whether an indexing method is available for this attribute or not. An unindexed attribute can be favored as a candidate in the grid, since rules that can replace the restrictions on this attribute with restrictions on indexed attributes can lead to large cost savings in semantic query optimization.

Historical data on query distribution is another source of information which can facilitate attribute selection. The rule of thumb in using this information is to select those unindexed attributes which are queried frequently and are updated infrequently. Choosing unindexed and frequently queried attributes leads to large cost savings. Choosing infrequently updated attributes allows extraction of rules that are valid for a longer period of time.

4.4. Range Selection for Continuous-Valued Attributes

The main issue in allowing continuous-valued attributes in the grid is the decomposition of the domain into a finite set of ranges for a grid coordinate. Factors such as the kinds of rules that can be learned, the cost of storing the grid, and the cost of searching it for rules play major roles in the way the attribute is decomposed. A coarse decomposition of an attribute reduces storage costs and the cost of scanning the grid for rules. However, such coarse decomposition is likely to abstract out some useful patterns in the data which will be overlooked by the discovery algorithm.

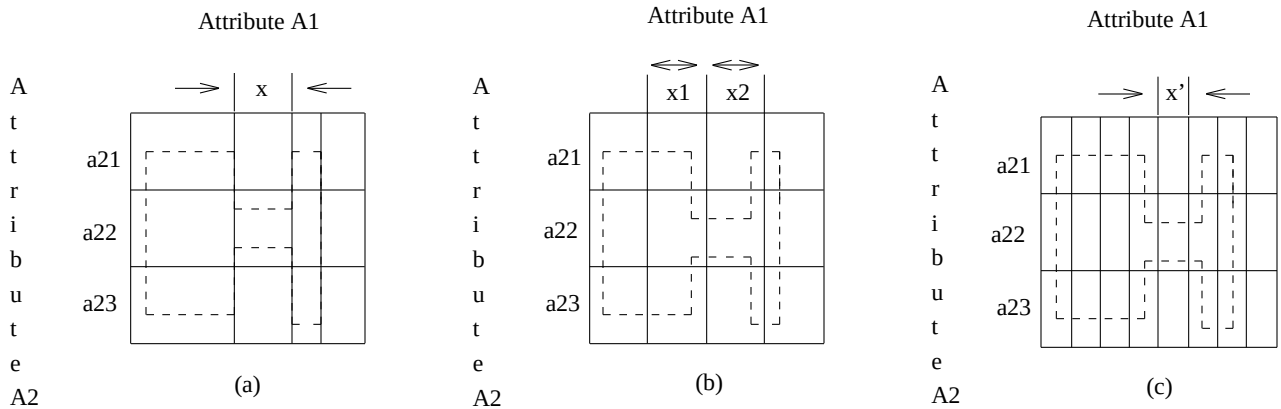


Fig. 4.: *Alternative range selections for a continuous-valued attribute*

A fine decomposition of the attribute increases the storage and scanning costs. Such decomposition learns many rules that may be weaker than coarse decomposition rules in two respects. One is that the rules might cover many small sub-ranges of an attribute which makes them applicable to a small set of data. Unless these rules can be merged to cover larger ranges, they may not be of much significance to semantic query optimization or other applications. The other weakness reveals itself in learning probabilistic rules. For such rules to be significant, they

have to hold true over data sets of certain sizes. If, due to fine decomposition of an attribute, only a few data points fall onto each grid, cell, the rules holding true over these grid cells usually will not pass statistical significance tests, due to inadequate number of data points. *Fig. 4* demonstrates an example grid with a continuous-valued attribute which is decomposed ideally, coarsely and finely in figures 4 a, b and c, respectively. The H-shaped closed region, in the figures, provides a distribution of the data in the grid. In the ideal case, the algorithm will discover a rule as follows:

$$(A1=x) \rightarrow (A2=a_{22}) \text{ (1).}$$

This rule will be overlooked in the coarse case, since the data is distributed across all values of attribute A2, when $A1=x_1$ and when $A1=x_2$. In the fine case, on the other hand, the following rule may be extracted:

$$(A1=x') \rightarrow (A2=a_{22}) \text{ (2).}$$

This rule is weaker than rule 1, since x' in figure 4c is a subrange of x in figure 4a. Hence, the fine decomposition, in this case, does not exploit the full pattern implied in the data.

5. Generation of Rule Closure

In this section, we first describe the domain of attributes and clauses representing restrictions on attributes. We then define and analyze a set of logical operations involving one or more restrictions. We also present a closure algorithm on the basic rules and analyze its correctness and completeness. A set of inference rules to derive new rules from basic constraints are also proposed and analyzed. We refer to these high-level or meta-rules as inference rules or closure rules.

5.1. Closure Rules

We restrict our discussion of closure rules to those rules involving ranges. The inference rules are based on logical implication, union, intersection and negation operations as follows. Let us assume we are given two basic correct and symmetric constraints:

$$IC_1: A1 \rightarrow C1$$

$$IC_2: A2 \rightarrow C2$$

Let us define the following ranges: $R_{100} = (0..100]$, $R_{150} = (100..150]$, $R_{200} = (150..200]$, $R_{250} = (200..250]$, $R_{300} = (250..300]$, $R_{350} = (300..350]$, $R_{\infty} = (350..\infty)$.

Closure Rule 1: Transitive Rule: Given IC_1 and IC_2 , if $C1 \rightarrow A2$ is symmetric and correct, then add $A1 \rightarrow C2$ to the rule set.

Proof of Correctness: Consider a tuple $t \in A1$. By $t \in A1$ we mean a tuple whose attribute values satisfy the clauses of the logical formula A. If $t \in A1$, then from IC_1 we can deduce $t \in C1$. Similarly, $t \in C1$ implies (due to $C1 \rightarrow A2$) $t \in A2$, and $t \in A2$ implies (due to $A2 \rightarrow C2$) $t \in C2$. Therefore, $(\text{for all } t): (t \in A1) \rightarrow (t \in C2)$. Thus, $A1 \rightarrow C2$.

Example: Given the rules

$(\text{DeadWt} \in R_{100}) \rightarrow (\text{ShipType} = \text{'Barge'})$,
 $(\text{ShipType} = \text{'Barge'}) \rightarrow (\text{CargoType} \in \{\text{'Grain'}, \text{'Metal'}\})$, and
 $(\text{CargoType} \in \{\text{'Grain'}, \text{'Metal'}\}) \rightarrow (\text{DollarValue} < 5000,000)$,

we can add the rule $(\text{DeadWt} \in R_{100}) \rightarrow (\text{DollarValue} < 5000,000)$ to the rule-set.

Closure Rule 2: Union Rule: Given IC_1 and IC_2 , if $A1$ and $A2$ are restrictions on the same attribute, say A , and $C1$ and $C2$ are restrictions on the same attribute say C , then add $\text{Union}(A1,A2) \rightarrow \text{Union}(C1,C2)$ to the rule-set.

Proof of Correctness: Consider a tuple $t \in \text{Union}(A1,A2)$. From IC_1 we know that if $t \in A1$, then $t \in C1$, and if $t \in C1$, then $t \in \text{Union}(C1,C2)$. On the other hand, if $t \in A2$, then using IC_2 we can deduce $t \in C2$, and thus $t \in \text{Union}(C1,C2)$. In either case, no matter which clause in the $\text{Union}(A1,A2)$ t belongs to, it will also belong to $\text{Union}(C1,C2)$. Hence, for all $t: [(t \in \text{Union}(A1,A2)) \rightarrow (t \in \text{Union}(C1,C2))]$. Therefore, $\text{Union}(A1,A2) \rightarrow \text{Union}(C1,C2)$.

Example: Given the rules

$(\text{DeadWt} \in R_{200}) \rightarrow (\text{ShipType} = \text{'SuperTanker'})$,
 $(\text{DeadWt} \in R_{250}) \rightarrow (\text{ShipType} = \text{'SuperTanker'})$,
 $(\text{DeadWt} \in R_{300}) \rightarrow (\text{ShipType} = \text{'SuperTanker'})$,
 $(\text{DeadWt} \in R_{\infty}) \rightarrow (\text{ShipType} = \text{'SuperTanker'})$,

we can add the rule $(\text{DeadWt} \geq 200) \rightarrow (\text{ShipType} = \text{'SuperTanker'})$ to the rule-set.

Closure Rule 3: Intersection Rule: Given IC_1 and IC_2 , add $\text{Intersection}(A1,A2) \rightarrow \text{Intersection}(C1,C2)$.

Proof of Correctness: Consider a tuple $t \in \text{Intersection}(A1,A2)$. If $t \in \text{Intersection}(A1,A2)$, then $t \in A1$. $t \in A1$ implies that $t \in C1$. $t \in \text{Intersection}(A1,A2)$ also implies that $t \in A2$, which in turn implies $t \in C2$. Thus from $t \in C1$ and $t \in C2$ we conclude $t \in \text{Intersection}(C1,C2)$. Therefore, (for all t): $[(t \in \text{Intersection}(A1,A2)) \rightarrow (t \in \text{Intersection}(C1,C2))]$. Therefore, $\text{Intersection}(A1,A2) \rightarrow \text{Intersection}(C1,C2)$.

Example: Given the rules

$(\text{DollarValue} > 3000) \rightarrow (\text{Issuer} \in \{\text{'Lloyds'}, \text{'Issuex'}, \text{'Issuery'}\})$,
 $(\text{ShipType} = \text{'SuperTanker'}) \rightarrow (\text{Issuer} \in \{\text{'Lloyds'}\})$

we can add the rule

$(\text{DollarValue} > 3000) \text{ AND } (\text{ShipType} = \text{'SuperTanker'}) \rightarrow (\text{Issuer} \in \{\text{'Lloyds'}\})$
to the rule-set.

Closure Rule 4: Complement Rule: Given IC_1 , add $B \rightarrow D$ to the rule-set, where $B = \text{Not}(C1)$ and $D = \text{Not}(A1)$.

Proof of Correctness: Consider a tuple $t \in \text{Not}(C1)$. Since t cannot satisfy $C1$, then $t \in A1$ will contradict the given implication in IC_1 . Thus, t cannot satisfy $A1$ and will belong to its complement (i.e. $t \in \text{Not}(A1)$).

Example: Given the rule

$(DeadWt > 150) \rightarrow (ShipType = 'SuperTanker')$,

we can add the rule

$Not(ShipType = 'SuperTanker') \rightarrow Not(DeadWt > 150)$

Assuming that the domain of attribute ShipType is {'Barge', 'Tanker', 'SuperTanker'}, $Not(ShipType = 'SuperTanker') = (ShipType \in \{'Barge', 'Tanker'\})$, and according to Table 3 $Not(DeadWt > 150) = (DeadWt \leq 150)$. Thus, the new rule will be as follows:

$(ShipType \in \{'Barge', 'Tanker'\}) \rightarrow (DeadWt \leq 150)$.

Proposition 4: Closure-rule application on symmetric rules results in new symmetric rules.

Proof: It can be verified for each closure rule using the definition of symmetric rules.

5.2. Closure Algorithm

In addition to the basic rule set, we include the following meta-rule in our rule-set which is based on the closed-world assumption:

$Union(Antecedents\ of\ Positive\ Rules) \rightarrow (All\ Tuples\ of\ the\ Universal\ Set)$

This rule states that the union of antecedents of the rules in the rule-set covers all the tuples that are represented in the grid. The rule implies that if a tuple is not included in the consequent of a rule, then it does not belong to the database. Usefulness of this meta rule is illustrated in the deriving of rule 4 in the next section "An Extended Example".

By applying the closure rules of section 5.1, to the basic query-transformation rules (QTR), defined in section 4.2., the following closure algorithm generates all possible new rules. Fig. 5

1. $S = \text{Basic Rule-set} + \text{meta-rule}$
2. For each pair $(R1, R2)$ of rules in S
 - 2.1. $S = S + (\text{Rules resulting from application of closure rules 1,2,3 and 4 to } (R1, R2))$.
3. Repeat step 2 until no more rules can be found.

Fig. 5.: Closure Algorithm

Next, we will discuss issues concerning the correctness and completeness of the closure algorithm. We define the closure algorithm to be correct, if the rules resulting from this algorithm constitute a set of correct rules.

Theorem 1: The closure algorithm is correct, i.e. if a query transformation rule QTR is derived from closure, then QTR is correct.

Proof: We show the correctness of the closure algorithm by the method of induction. At each iteration i , the algorithm adds new rules to the set S_{i-1} to obtain the rule-set S_i , where S_i and S_{i-1} are the rule-sets at the end of the i th and the $(i-1)$ th iterations. Thus, the set S can be regarded as a sequence $\{S_0, S_1, \dots, S_n\}$ of all intermediate rule-sets S_i , where S_0 is the basic rule-set, S_n is the final rule-set and $0 \leq i \leq n$.

Base Step: S_0 constitutes a correct set of rules. In previous sections we demonstrated the correctness of the basic rules resulting from scanning the grid. Thus, S_0 consists of a set of correct rules.

Inductive Step: If S_i constitutes a correct set of rules, then S_{i+1} also constitutes a correct set of rules. S_{i+1} is the result of adding new rules to S_i at the $(i+1)$ th iteration. The new rules are added by applying the closure rules 1,2,3 and 4 to the pairs of rules in S_i . Since we have shown that the closure rules are correct (i.e. their application results in correct rules), and since S_i is correct, we conclude that S_{i+1} constitutes a correct set of rules. This concludes the correctness of the closure algorithm.

The grid algorithm is incomplete in general, namely there are rules that will not be discovered by the grid and its algorithm. There may exist patterns in the data that will not be apparent in the chosen grid. These include patterns that do not coincide with the grid orientation or patterns that are overlooked due to the resolution of the grid (an example of this is provided in a later section). We will not discuss a closure on the attribute set and will only be considering the patterns among a limited set of attributes. This will leave many other rules associated with other attribute pairs unexposed. Completeness of the grid and its integrity constraint learning algorithm can be shown in a limited sense, however. For any rule $A \rightarrow C$ in the derived rule-set DRS, the closure procedure learns a corresponding rule $A \rightarrow C'$, such that $(A \rightarrow C)$ and $(A \rightarrow C')$ are equivalent rules in the data distribution.

Theorem 2: The closure algorithm is complete, i.e. if a rule $R=(A \rightarrow C)$ belongs to the derived-set DRS of a basic rule-set BRS, then the closure algorithm will discover an equivalent rule $A \rightarrow C'$.

Proof: Let DRS be the derived rule set. We note that the antecedent sets $AS()$ and consequent sets $CS()$ of BRS and DRS are related. Using definition of derived set, we know that $A \in AS(DRS) = \text{range-set-closure}(AS(BRS))$. Thus, there is a subset $S1=(A1, A2,...,Ak)$ of $AS(BRS)$ for rule R , and a derivation f , such that $f:S1 \Rightarrow A$, where ' $\Rightarrow$ ' represents derivation. Let us consider the set $S2=\{ C_i \mid (A_i \rightarrow C_i \in BRS) \text{ and } A_i \in S1 \}$, and derive expression C' from the same derivation f with substitution of A_i by C_i (i.e. $f:S2 \rightarrow C'$). We note that the rule $A \Rightarrow C'$ will be derived from our closure procedure, due to the construction of C' . Furthermore, $A \rightarrow C$ is symmetric since it is a member of the derived rule set, and $A \rightarrow C'$ is a symmetric since it is derived from the closure algorithm. Hence $R_x(C)=R_x(A)=R_x(C')$, or rules $A \rightarrow C$ and $A \rightarrow C'$ are equivalent in the given data distribution.

Complexity Analysis:

Next, we provide an analysis of the complexity of the proposed data-driven framework. As was demonstrated in *Fig. 1*, we start by selecting a set of attributes, out of the n existing attributes, for the formation of the grid. There are $O(2^n)$ possible choices for attribute selections. However, assuming that we form only 2-dimensional grids, this number can be reduced to $O(n^2)$. The second step is concerned with the formation of the grid. This requires one scan through all T tuples in the database. The third step is concerned with the derivation of basic rule-set from the number of cells, C , in the grid. In general, one scan through the grid is needed for every possible antecedent. The antecedent set for basic rule sets includes only the coordinate-range antecedents. Thus, the cost of deriving the basic rule-set is in the order of $O(C*N)$, where N = number of Antecedents for the basic rule set. The fourth step is concerned with the computation of the closure on the set of N basic rules. The complexity of this algorithm is of the order $O(N*2^c)$, since the number of consequents are limited by 2^c . The number of rules in the closure set is limited by the product of the cardinalities of antecedent and consequent ranges. The fifth step involves pruning the set of R rules resulting from the closure algorithm. The complexity of

this step is in the order $O(R)$, assuming that the evaluation of the proposed rules by the conventional query optimizer takes constant time c .

We note that the total complexity of deriving the final rule-set is linear in terms of the number of tuples in the database. However, we note that this complexity is exponential in terms of number of attributes and the number of attribute values represented in the database. When the number of attributes and the number of attribute values are small, the steps involved in the framework can be carried out without external help in a reasonable amount of time. For large numbers of attributes and attribute values, however, the external help from the DBA might be required to reduce the complexity of carrying out the above steps.

6. An Extended Example

In this section, we provide a sample session to illustrate the working of the proposed grid based learning algorithm. Let us consider the shipping database as discussed in the introduction. *Table 7* gives the rules that will be used as example query transformation rules. Furthermore, we will relax the symmetry constraint on the consequent by dropping the clause representing the antecedent from the consequent of the rules to simplify the expressions. The relaxation does not affect the correctness of query transformation rules.

Rule #	Antecedent	Implication	Consequent
R_1	$(DeadWt > 350)$	$\rightarrow$	$(FacilityType = 'OffShore')$
R_2	$(DeadWt > 300)$	$\rightarrow$	$(Business = 'Leasing')$
R_3	--	--	$(Coverage \leq DollarValue)$
R_4	--	--	$(Quantity \leq Capacity)$
R_5	$(CargoType \in \{'Natural Gas', 'Refined'\})$ and $(DollarValue > 500)$	$\rightarrow$	$(Facilitytype = 'General')$
R_6	$(DeadWt > 150)$	$\rightarrow$	$(ShipType = 'SuperTanker')$
R_7	$(DollarValue > 3000)$ and $(ShipType = 'SuperTanker')$	$\rightarrow$	$(Issuer = 'Lloyds')$
R_8	$(Business = 'Petroleum')$	$\rightarrow$	$(CargoType \in \{'Natural Gas', 'Refined', 'Oil'\})$

Table 7.: Semantic Integrity Constraints.

6.1. Learning Query Transformaion Rules

Rule 6: (*Table 7*) The semantic knowledge of a constraint represented in this grid is: "All vessels weighing more than 150 are super tankers." The equivalent constraint to be learned is: $(DeadWt \geq 200) \rightarrow ShipType = 'SuperTanker'$. Scanning the rows of the grid shown in Fig. 6 will result in the following simple rules:

- (1) $(DeadWt \in R_{100}) \rightarrow (ShipType = 'Barge')$
- (2) $(DeadWt \in R_{150}) \rightarrow (ShipType \in \{'Barge', 'Tanker'\})$
- (3) $(DeadWt \in R_{200}) \rightarrow (ShipType = 'SuperTanker')$
- (4) $(DeadWt \in R_{250}) \rightarrow (ShipType = 'SuperTanker')$
- (5) $(DeadWt \in R_{300}) \rightarrow (ShipType = 'SuperTanker')$
- (6) $(DeadWt \in R_{350}) \rightarrow (ShipType = 'SuperTanker')$
- (7) $(DeadWt \in R_{\infty}) \rightarrow (ShipType = 'SuperTanker')$

		ShipType		
		Barge	Tanker	SuperTanker
D e a d W t	R_{100}	h	0	0
	R_{150}	f	e	0
	R_{200}	0	0	b
	R_{250}	0	0	c
	R_{300}	0	0	a
	R_{350}	0	0	d
	R_{∞}	0	0	g

Fig. 6.: A data-distribution grid representing attributes (Dead Weight, Ship Type).

Taking a union of rules 3 through 6 will result in the following rule: $Union((DeadWt \in R_{200}), (DeadWt \in R_{250}), (DeadWt \in R_{300}), (DeadWt \in R_{350})) \rightarrow (ShipType = 'SuperTanker')$. The disjuncts in the antecedent of this rule will be aggregated into a range as a result of the union operation which results in the following rule: $(DeadWt \geq 200) \rightarrow (ShipType = 'SuperTanker')$

Rule 8: (Table 7) The semantic knowledge of a constraint (among others) represented in this grid is: "A petroleum business's cargo type is either natural gas, oil or refined oil." The equivalent constraint to be learned is: $(Business = 'Petroleum') \rightarrow (CargoType \in \{'NaturalGas', 'Refined', 'Oil'\})$. Scanning the rows of the grid in Fig. 7 will result in the following basic rules, one of which is the desired rule about petroleum and cargo types.

- $(Business = 'Grain') \rightarrow (CargoType \in \{'Wheat', 'Rice', 'Barley'\})$
- $(Business = 'Petroleum') \rightarrow (CargoType \in \{'NaturalGas', 'Refined', 'Oil'\})$
- $(Business = 'Metal') \rightarrow (CargoType \in \{'Copper', 'Iron'\})$

Rule 4: (Table 7) The semantic knowledge of a constraint represented in the grid shown in Fig. 8 is: "The quantity is always less than the capacity". The equivalent constraint to be learned is: $(Quantity < Capacity)$. Fig. 8 displays two alternatives of representing the distribution space. Let us denote quantity by Q and capacity by C.A. The boundary that constrains all the data items in

		CargoType							
		Wheat	Rice	Barley	Nat. Gas	Refined	Oil	Copper	Iron
Business	Grain	a	b	c	0	0	0	0	0
	Petroleum	0	0	0	d	e	f	0	0
	Metal	0	0	0	0	0	0	g	h

Fig. 7.: A data-distribution grid representing attributes (Business,Cargo Type).

the grid in one region corresponds to the line of equation $C=Q$. The learning of this constraint is complicated by the fact that the linear boundary of line $C=Q$ does not coincide with the grid lines, in an orthogonal orientation. The learning of this rule is facilitated by an oblique grid $\langle Q+C, Q-C \rangle$.

- (1) $(0 < C-Q \leq k) \rightarrow (3k < Q+C \leq 4k) \text{ or } (4k < Q+C \leq 5k) \text{ or } (5k < Q+C \leq 6k)$
- (2) $(k < C-Q \leq 2k) \rightarrow (2k < Q+C \leq 3k) \text{ or } (3k < Q+C \leq 4k) \text{ or } (4k < Q+C \leq 5k) \text{ or } (5k < Q+C \leq 6k)$
- (3) $(2k < C-Q \leq 3k) \rightarrow (2k < Q+C \leq 3k) \text{ or } (3k < Q+C \leq 4k) \text{ or } (4k < Q+C \leq 5k)$
- (4) $(3k < C-Q \leq 4k) \rightarrow (3k < Q+C \leq 4k) \text{ or } (4k < Q+C \leq 5k)$

We note that meta-rule, representing the closed-world assumption, implies that the data in the consequent of the above rule includes all tuples t , namely

for all t , $((0 < C-Q \leq k) \text{ or } (k < C-Q \leq 2k) \text{ or } (2k < C-Q \leq 3k) \text{ or } (3k < C-Q \leq 4k))$.

The union of the above intervals leads to the following:

for all t , $(0 < C-Q \leq 4k)$ which implies

for all t , $(Q < C < Q+4k)$ which implies

for all t , $(Q < C)$ and for all t , $(C < (Q+4k))$

Thus, we can conclude that all the data items have fallen onto the positive side of the $C-Q=0$ line, namely $C > Q$.

6.2. Use in Semantic Query Optimization

Now, let us consider a query expressed in a Prolog like syntax, to be,

$Q_0: (DeadWt > 400) \text{ and } (DollarValue > 4000): (? Destination)$

The first step in semantic query optimization is to generate the space of semantically equivalent queries. This is done by applying rules R_1 through R_8 to Q_0 . Actually only rules R_1, R_2, R_6 and R_7 are applicable, giving rise to queries Q_1 through Q_{15} , which were all semantically equivalent to Q_0 .

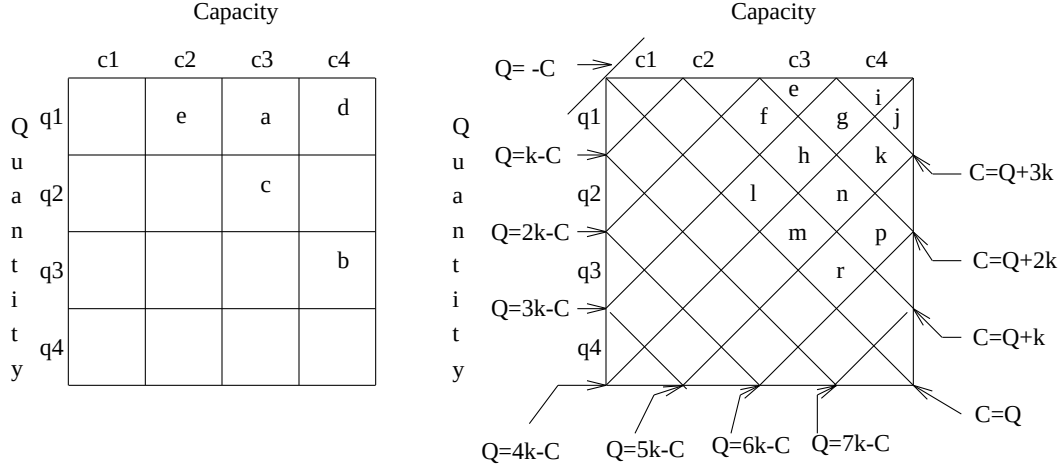


Fig. 8.: An orthogonal and an oblique grid representing attributes Quantity and Capacity.

Fig. 9 shows the entire space generated, in which each node represents a semantically equivalent but syntactically distinct query. The clauses C_1 through C_6 are as follows,

- C_1 : (DeadWt > 400)
- C_2 : (ShipType = 'SuperTanker')
- C_3 : (DollarValue > 4000)
- C_4 : (FacilityType = 'OffShore')
- C_5 : (Business = 'Leasing')
- C_6 : (Insurer = 'Lloyds')

The transitions between nodes in Fig. 9 represent the transformations carried out, by rule applications. Two basic inference rules, called *clause introduction (CI)* and *clause elimination (CE)* are used to carry out the transformations.

Let,

A, B be distinct clauses,
and, $(A \rightarrow B)$ be a rule.

Now,

(CI): $(A) \text{ and } (A \rightarrow B) \equiv (A) \text{ and } (B) \text{ and } (A \rightarrow B)$

(CE): $(A) \text{ and } (B) \text{ and } (A \rightarrow B) \equiv (A) \text{ and } (A \rightarrow B)$

Rules like $(A \rightarrow B)$ are integrity constraints of the system are all true in the current state of the database. Thus, we are only interested in the clauses $A, B, A \text{ and } B$, etc. Each of the queries Q_0 through Q_{15} may be optimized by a conventional optimizer. The optimizer generates a number of query execution strategies (called query plans) and then attaches an estimated cost with each by using a cost model based on the the expected number of CPU instructions and page fetches[38]. The execution cost estimates provided by the optimizer are dependent on the system configuration and thus we do not give any units. However, for our present purpose only their relative values are important. The data is shown in Table 8. Thus, the semantic query optimization can yield a plan with execution cost of 128 units whereas the best plan for original query without semantic query optimization had a cost of 605 units.

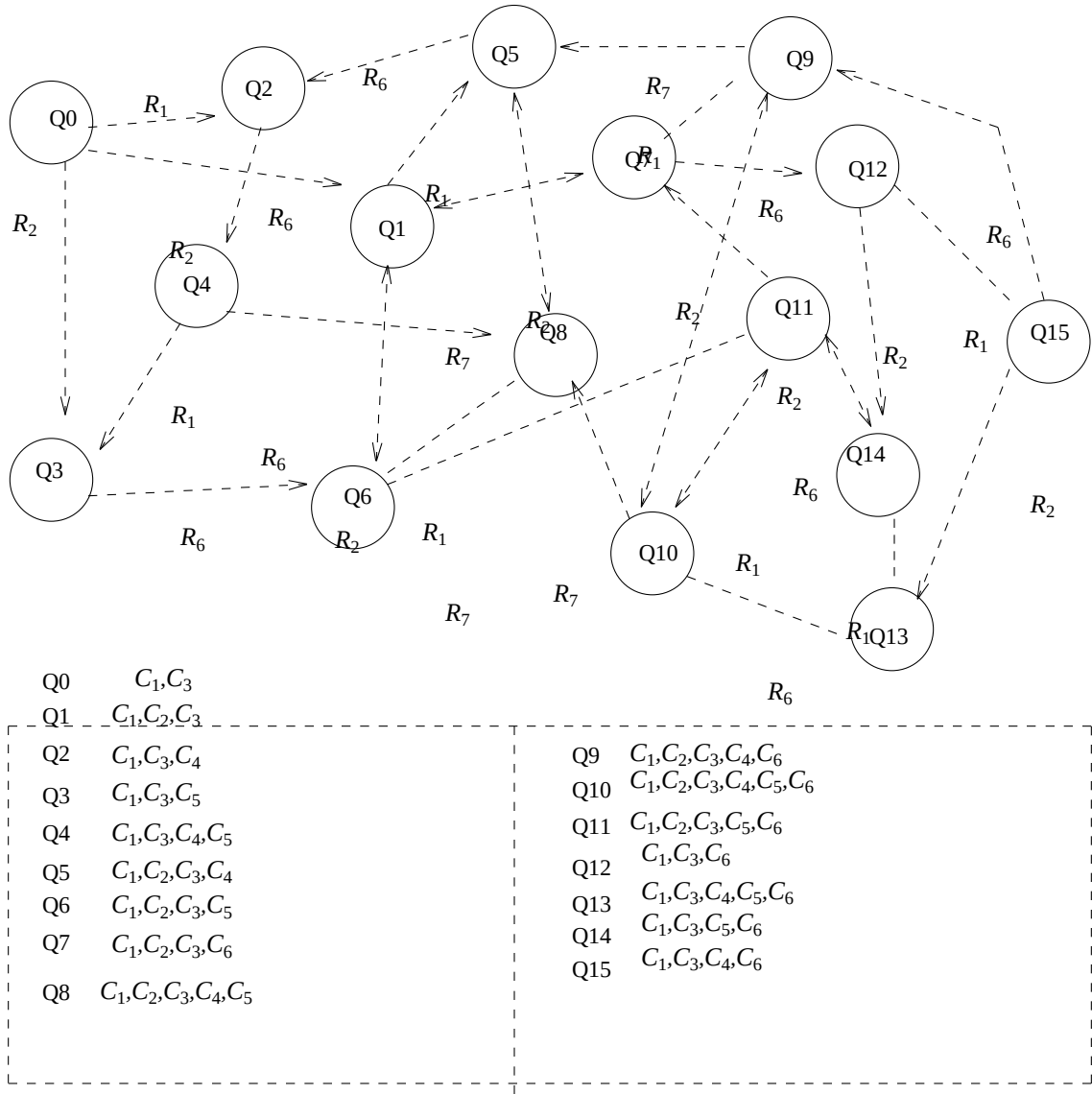


Fig. 9.: Space of Semantically Equivalent Queries.

Query#	0	1	2	3	4	5	6	7
Total Cost	605.7	157.6	605.7	127.8	127.8	157.6	127.0	157.6
Query#	8	9	10	11	12	3	14	15
Total Cost	127.0	157.6	127.0	127.0	227.6	127.8	127.8	227.6

Table 8.: *Execution Costs for Various Queries*

7. Conclusions

We have introduced a data-driven framework for the automatic discovery of query transformation rules in semantic query optimization. Based on this framework, we have introduced a grid representation as the data distribution in the space of a set of finite attributes. We have provided an algorithm for extracting a set of query transformation rules. We have shown the correctness of our algorithm and the rules derived from it. We also proved the completeness of our algorithm in a limited sense. Finally, an extended example of a semantic query optimization session was provided to demonstrate our discovery framework.

As part of our future research, we would like to extend our framework to the discovery of statistical rules for processing statistical queries. Lastly, we would like to investigate the benefits and trade-offs of incorporating user-supplied domain knowledge and other sources of information such as query distribution information into our framework. In particular, we plan to integrate our work with query-driven approaches, in order to monitor the performance of the discovered query transformation rules during actual operation of the database. We would like to utilize the heuristics for semantic query optimization [1] for selecting the attribute sets to create grids. Furthermore we would like to characterize the set of oblique grids which may yield useful query transformation rules.

8. Acknowledgements

This research was supported by the graduate school of University of Minnesota, and the Minnesota Department of Transportation. Prof. Raymond Mooney (Univ. of Texas) helped us with the implementation of the traditional AI learning algorithms including AQ and ID3. The project benefited from the interactions with the discovery in database group within the computer science department, at the University of Minnesota.

9. References

1. J. J. King, QUIST : A system for semantic query optimization in relational databases, *Proc. 7th VLDB Conf.*, (1981).
2. Hammer and Zdonik, Knowledge Based query processing, *Proc. 6th Conf. on VLDB*, (1980).
3. S .T. Shenoy and Z. M. Ozsoyoglu, Design and Implementation of a Semantic Query Optimizer, *IEEE Transactions on Knowledge and Data Engineering*, pp. 362-375 (1989).
4. H. H. Pang, H. J. Lu, and B. C. Ooi, An Efficient Semantic Query Optimization Algorithm, *Proc. Data Engineering Conference*, IEEE, (1991).
5. G. M. Lohman, Panel Discussion on Semantic Query Optimization, *Proc. Data Engineering Conf.*, (1985).

6. S. Shekhar, J. Srivastava, and S. Dutta, A Formal Model of Trade-offs between Execution and Optimization Costs in Semantic Query Optimization, *Intl. Conf. on Very Large Databases (VLDB)*, (1988).
7. C. T. Yu and W. Sun, Automatic Knowledge Acquisition and Maintenance for Semantic Query Optimization, *IEEE Transactions on Knowledge and Data Engineering*, pp. 362-375 (1989).
8. S. T. Shenoy and Z.M.Ozsoyoglu, A System for Semantic Query Optimization, *Proc. ACM-SIGMOD*, pp. 181-195 (1987).
9. A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *The Design and Analysis of Computer Algorithms*, Bell Telephone Laboratories (1974).
10. J. Zytkow and J. Baker, Interactive Mining of Regularities in Databases, *Knowledge Discovery in Databases*, The AAAI Press, (1991).
11. M. Siegel, E. Sciore, and S. Salveter, Rule Discovery for Query Optimization, *Knowledge Discovery in Databases*, The AAAI Press, (1991).
12. M. Siegel, *Automatic Rule Derivation for Semantic Query Optimization. Ph.D. diss.*, Boston Univ. (1988).
13. P. Langley, J. Zytkow, H. Simon, and G. Bradshaw, Rediscovering Chemistry with the BACON System, pp. 307-330 in *Machine Learning: An Artificial Intelligence Approach*, ed. T. M. Mitchell, Tioga, Palo Alto, California (1983).
14. C. L. Chang, DEDUCE 2: Further Investigations of Deduction in Relational Data bases, pp. 201-236 in *Logic and Data Bases*, ed. J. Minker, Plenum Press, New York (1978).
15. J. M. Nicolas, Logic for Improving Integrity Checking in Relational Databases, *Acta Informatica* **18** pp. 227-253 Springer Verlag, (1982).
16. R. S. Michalski, A Theory and Methodology of Inductive Learning, in *Machine Learning: An Artificial Intelligence Approach*, ed. T. M. Mitchell, Morgan Kaufmann Publishers, Inc., Los Altos, California (1986).
17. R. S. Michalski and R. E. Stepp, Learning from Observation: Conceptual Clustering, pp. 331-363 in *Machine Learning: An Artificial Intelligence Approach*, ed. T. M. Mitchell, Tioga, Palo Alto, California (1983).
18. J. R. Quinlan, Probabilistic Decision Trees, pp. 140-152 in *Machine Learning: An Artificial Intelligence Approach*, ed. Yves Kodratoff, Morgan Kaufmann Publishers, Inc., San Mateo, California (1990).
19. S. C. Shapiro, *Encyclopedia of Artificial Intelligence*, A Wiley-Interscience Publication (1990).
20. R. R. Sokal and R. H. Sneath, *Principles of Numerical Taxonomy*, W. H. Freeman, San Francisco (1963).
21. R. M. Cormark, A review of classification, pp. 134-321 in *J. Roy. Stat. Soc., Series A*, (1971).
22. M. R. Anderberg, *Clustering analysis*, Academic Press, New York (1973).
23. J. C. Gower, A comparison of some methods of cluster analysis, *Biometrics* **23**, pp. 623-637 (1967).
24. E. Diday and J. C. Simon, Clustering analysis, *Communication and Cybernetics*, Springer-Verlag, New York, (1976).
25. R. S. Michalski, Knowledge acquisition through conceptual clustering: A theoretical framework and an algorithm for partitioning data into conjunctive concepts, *J. Pol. Anal. Inform. Sys* **4**, pp. 219-244 (1980).
26. P. Langley and S. Sage, Conceptual Clustering as Discrimination Learning, *Proceedings of the Fifth Biennial Conference of the Canadian Society for Computational Studies of Intelligence*, pp. 95-98 (1984).
27. D. Fisher, *A Hierarchical Conceptual Clustering Algorithm*, Technical Report, Department of Information and Computer Science, University of California, Irvine (1984).
28. S. J. Hanson, Conceptual Clustering and Categorization: Bridging the Gap between Induction and Causal Models, pp. 235-268 in *Machine Learning: An Artificial Intelligence Approach*, ed. Yves Kodratoff, Morgan Kaufmann Publishers, Inc., San Mateo, California (1990).
29. S. P. Ghosh, Statistics Metadata: Linear Regression Analysis, pp. 3-17 in *Foundations of Data Organization*, ed. Katsumi Tanaka, Plenum Press, New York (1987).

30. D. B. Lenat, The Role of Heuristics in Learning by Discovery: Three Case Studies, in *Machine Learning: An Artificial Intelligence Approach*, ed. T. M. Mitchell, Tioga, Palo Alto, California (1983).
31. P. Langley, J. Zytkow, H. Simon, and G. Bradshaw, The Search for Regularity: Four Aspects of Scientific Discovery, pp. 425-469 in *Machine Learning: An Artificial Intelligence Approach*, ed. T. M. Mitchell, Morgan Kaufmann Publishers, Inc., Los Altos, California (1986).
32. R. Elmasri and G. Wiederhold, Data Model Integration Using the Structural Model, *SIGMOD*, (1979).
33. M. Hammer and D. McLeod, Semantic Integrity in a Relational Data Base System, *VLDB*, (1975).
34. K. Eswaran and D. D. Chamberlin, Functional Specifications of a Subsystem for Database Integrity, *VLDB*, (1975).
35. E. Codd, Extending the Database Relational Model to Capture More Meaning, *TODS* **4:4**(December 1979).
36. P. Chen, The Entity Relationship Mode-Toward a Unified View of Data, *TODS* **1:1**(March 1976).
37. J. Schmidt and J. Swenson, On the Semantics of the Relational Model, *SIGMOD*, (1975).
38. L. F. Mackert and G.M.Lohman, R* Optimizer Validation and Performance Evaluation for Local Queries, *Proc. ACM-SIGMOD*, pp. 84-95 ACM, (1986).